©VIT IPR&TTCELL

1. Title of the invention:

VITAL — IoT-Enabled Smart Health Monitoring and Emergency Alert System with AI-Based Anomaly Prediction for Smart City Integration

2. Field /Area of invention:

This invention falls under embedded IoT systems for continuous health monitoring. It measures heart rate, blood-oxygen saturation (SpO_2) and body temperature, checks whether each reading can be trusted, learns what is normal for each person using machine learning, and warns early when a reading is drifting towards danger. Alerts are escalated in steps depending on how serious the situation is, and anonymised area-level health data can be shared with a smart-city platform.

3. Prior Patents and Publications from literature (provide a table summarizing the prior art)

A. Prior patent literature

Reference / citation	What the work discloses	Relevance to VITAL	Technical gap / difference
P1. US 8,909,330 B2 — Body-worn device and associated system for alarms/alerts based on vital signs and motion (Sotera Wireless Inc., 2014)	A monitor worn on the body tracks vital signs and also works out what the wearer is doing (resting, walking, falling). Alarms come from fixed outer limits plus inner limits based on the patient's recent values. When movement spoils the readings, rule-based logic holds the alarm back. Data goes wirelessly to a nursing station.	Of all the references we found, this one is closest to VITAL. It also monitors several vitals continuously, handles readings spoiled by motion, raises alarms and lets a supervisor watch remotely.	Its limits are either preset or based on a short recent period. It does not build a long-term personal model from only the good-quality data, it does not estimate how long until a limit is reached, and it has no guardian messages or anonymised ward-level sharing.
P2. US 2016/0302671 A1 — Prediction of health status from physiological data (Microsoft Technology Licensing LLC, 2016)	A wearable optical sensor reads heart rate and breathing rate. Resting values are compared with the user's baseline from the previous few days. If the change passes a fixed or percentage threshold, classifiers (Gaussian mixture, discriminant analysis, SVM) estimate the health state and the chance of a fever coming on. With permission, results can be shared with caregivers.	Like VITAL, it builds a personal baseline and uses machine learning to spot illness early, and it can tell caregivers.	Its baseline works on day-level resting values, and it only throws away values that are physically impossible. Nothing checks the quality of each reading before it goes into the baseline. It also doesn't combine SpO_2 and temperature on one shared device, give a live time-to-threshold, or share data with a city platform in stages.
P3. US 11,922,796 B2 — Predicting critical alarms (Koninklijke Philips N.V., 2024)	Once a non-critical alarm has lasted a few seconds, a machine-learning model (for example boosted decision trees) looks at the last 1–3 minutes of heart rate, breathing rate, SpO_2 and heart-rate variability. It predicts whether a critical alarm will follow in roughly the next minute and alerts clinicians if so.	It tries to catch deterioration before the critical alarm happens, which is the same goal as our trend warning.	The prediction only starts after an alarm already exists, and it learns from past hospital alarm data over a very short horizon. There is no personal baseline, no easy-to-explain time-to-threshold, and it is not a low-cost home device with guardian and city alert levels.
P4. US 12,125,597 B2 — Systems and methods for contagious illness surveillance and outbreak detection (Kinsa Health LLC, 2024)	Connected smart thermometers send temperature readings with location to a central server. The server counts fevers in each region, looks for unusual rises that may mean an outbreak, and gives the combined figures to public-health bodies through APIs and dashboards.	It groups home-device readings by region for public health, which is basically what our smart-city layer does.	Here each user's readings and location are stored centrally, and only one parameter is measured, now and then. It has no emergency escalation for individuals, no personal anomaly detection, no merging of small regions, and no rule that identity leaves the home hub only for a consented critical alert.

B. Research publications

Reference / citation	What the work discloses	Relevance to VITAL	Technical gap / difference
R1. S. Deshmukh, S. Shah, A. Wahedna and N. Sabnis (2025), "IoT-enabled smart healthcare system with machine learning for real-time vital sign monitoring and anomaly detection," Indonesian Journal of Electrical Engineering and Computer Science, 39(2), 1155–1163. DOI: 10.11591/ijeecs.v39.i2.pp1155-1163.	An IoT system built on the MAX30102 streams vital signs and uses an LSTM network to predict upcoming values and detect anomalies.	Same type of PPG sensor as ours, also with ML-based prediction and anomaly detection.	The LSTM is a general time-series model. There is no robust baseline for each person, no quality check before training, no time-to-threshold and no alert or data-sharing levels.
R2. P. Arpaia, F. Crauso, E. De Benedetto, L. Duraccio, G. Improta and F. Serino (2022), "Soft transducer for patient's vitals telemonitoring with deep learning-based personalized anomaly detection," Sensors, 22(2), 536. DOI: 10.3390/s22020536.	A wearable telemonitoring platform that trains an LSTM autoencoder separately for each patient to find anomalies. The authors report over 93% accuracy.	It shows that anomaly detection tuned to each patient works on vital signs, which supports our approach.	It needs a deep model per patient. It does not check signal quality before training, has no simple trend projection with time-to-threshold, does not handle several users sharing one device, and has no privacy levels for city data.
R3. R. Ahmad et al. (2022), "Development of blood oxygen level, heart rate and temperature monitoring system by using ESP32," Proc. 4th Int. Conf. on Smart Sensors and Application (ICSSA). DOI: 10.1109/ICSSA54161.2022.9870943.	A home-monitoring prototype on the ESP32 with a MAX30102 and an infrared temperature sensor. Its accuracy was checked against a hospital patient monitor.	The hardware is almost the same as ours, so it is a useful baseline for what the sensors can do.	It only measures and shows the data remotely. There is no learning, prediction, emergency escalation or city link.

Where the prior art stops: Going through these references, we found that most of the individual pieces already exist somewhere. Wearable and IoT vital monitors, threshold alarms, ignoring alarms during motion, personal baselines, ML anomaly detection, short-term alarm prediction, caregiver alerts and regional grouping of home readings have all been done, alone or in partial combinations. What none of them do is link the steps the way VITAL does. In VITAL, every reading first goes through a quality check. Only readings that pass are tagged to whoever is logged in on the shared device and used to train that person's model. Each 10-second window is then compared with both the person's own per-vital baseline and an Isolation Forest. A steady-trend check estimates how long until a limit is crossed. Finally, the severity decides the alert level, and the amount of data shared grows only as severity grows: from the device itself, to a guardian, to a supervisor, and lastly to an anonymised ward-level city summary.

4. Summary and background of the invention (Address the gap / Novelty)

Background and Gap

Most cheap health monitors, and most of the student and research IoT projects we looked at, measure heart rate, SpO_2 and temperature and compare each value with fixed limits meant for the general population, such as heart rate above 100 bpm or SpO_2 below 95%. Everyone gets the same limits. So if someone's resting pulse goes from 58 to 80 bpm, that is a big change for them, but the device still calls it "normal". Meanwhile, a person whose usual pulse sits close to a limit keeps getting alarms for nothing.

There is a second problem. Optical (PPG) readings get spoiled easily when the finger moves or isn't placed properly. Use those readings directly and you get false alarms. Use them to learn someone's baseline and the baseline quietly goes wrong. On top of that, most devices only alarm after a value has already crossed the limit, the alert stays on the device, and if the data is shared for public health at all, it usually goes out with the person's identity attached.

VITAL handles this in stages. Every one-second reading first goes through a signal-quality model. Readings that fail are not shown as valid and are not learned from. The ones that pass are linked to whoever started the session on the shared device and grouped into 10-second windows. Each window is compared with that user's own model, which has two

parts: a robust baseline for each vital and an Isolation Forest that looks at all features together. At the same time, a trend check estimates when a vital will cross a danger limit, if it keeps going the way it is.

These three things (rules for sustained limits, the personal anomaly score and the trend projection) together decide how serious the situation is, and that decides the alert level. Each level tells more people and shares a bit more data. First the local display and the user page, then a Telegram message to a guardian, then the supervisor station. For population health, an anonymised ward-level summary goes to a city platform over MQTT. Small groups are merged, and the person's identity goes to emergency services only for a critical alert they have agreed to in advance.

Novelty Statement

What is new in VITAL is not any single sensor, model or alert channel. It is the way quality checking, personal learning, trend projection and step-wise data sharing are tied together. Signal quality decides both what is shown and what is learned. Each user gets their own model even though they share one device, because readings are tied to the logged-in session. Anomalies are judged against that user's own baseline and pattern. A trend is only projected if it is steady, so a single jump doesn't cause a warning. And the same severity value decides both who gets alerted and how much data is shared. The references we cited each cover some of this (monitoring, personal baselines, alarm prediction, regional grouping), but none of them describe the full chain.

For our bench-scale proof of concept, we built this with an ESP32 DevKit V1, a MAX30102 pulse-oximetry module, a GY-906 (MLX90614) infrared thermometer, a 0.96-inch SSD1306 OLED, a laptop running the server and a web dashboard. We also wrote a device simulator that sends exactly the same data format as the firmware. This let us test the whole pipeline, including dangerous cases like low oxygen that we obviously can't create safely on a person.

5. Objective(s) of Invention

1. **Build a low-cost IoT monitor that keeps measuring heart rate, SpO_2 and body temperature and shows them on the device and on a live web dashboard.**
2. **Check the quality of every reading, and keep readings spoiled by movement or bad finger placement out of the display, the alerts and the learning.**
3. **Link readings from a shared device to the person logged in, so that each user gets their own personal model.**
4. **Catch readings that are unusual for that particular person, using a robust per-vital baseline together with an Isolation Forest.**
5. **Warn before a danger limit is crossed, by projecting steady trends and estimating how much time is left.**
6. **Escalate step by step (local, guardian, supervisor, city) and share more data only when the situation is more serious.**
7. **Give a smart-city platform ward-level health figures without sending anyone's identity, except in emergencies the user has agreed to.**

6. Working principle of the invent (in brief)

1. **The user logs in on the dashboard and starts a session. From then on, the shared device belongs to that user.**
2. **The ESP32 reads the red and infrared channels of the MAX30102 (100 Hz, averaged down to 25 samples/s) and the skin temperature from the MLX90614. It works out heart rate and SpO_2 and sends one packet every second over Wi-Fi.**
3. **The server gives each reading a quality score. Anything below the cut-off is thrown out.**
4. **For readings that pass, skin temperature is turned into body temperature using the user's own offset, and the readings are grouped into 10-second windows.**
5. **Each window is compared with the user's personal model (per-vital baseline plus Isolation Forest), giving an anomaly score from 0 to 1.**
6. **A straight line is fitted to the last 10 minutes and extended 15 minutes ahead. If it crosses a danger limit and the rise is steady, the time left is estimated.**
7. **The rules, the anomaly score and the projection together give the severity: normal, watch, early warning or critical.**
8. **The severity picks the alert level: device and user page, Telegram to a guardian, alert at the supervisor station, and the anonymised ward summary for the city.**

9. Good-quality windows are saved to the user's history, and the personal model is retrained from time to time.

In our prototype, the server runs on a laptop and anyone on the same network can open the dashboard in a browser. For dangerous conditions, we use the device simulator, which sends the same message format as the ESP32 firmware.

7. Description of the invention in detail (Include drawing and or photograph as needed)

VITAL works as a chain of stages. A low-cost embedded device takes the readings, the server checks whether they are reliable, learns what is normal for each user, looks for signs of things getting worse and raises alerts based on how serious it is. At each level, only as much personal data is shared as is actually needed.

7.1 System architecture

Our implementation has these parts:

- an ESP32 DevKit V1 microcontroller (dual core, built-in Wi-Fi);
- a MAX30102 red/infrared PPG module for heart rate and SpO_2;
- a GY-906 (MLX90614) infrared thermometer that reads skin temperature without contact;
- a 0.96-inch SSD1306 OLED that shows readings and alerts on the device;
- a server with a database, the signal-quality, personal-model and trend modules, and the alert service;
- a web dashboard with user, supervisor and city views; and
- Telegram (through the Bot API) for messages, and MQTT for sending data to the city platform.

The overall flow is: Sensors → Signal-quality gate → User-attributed windows → Personal anomaly score and trend projection → Severity → Escalation tier.

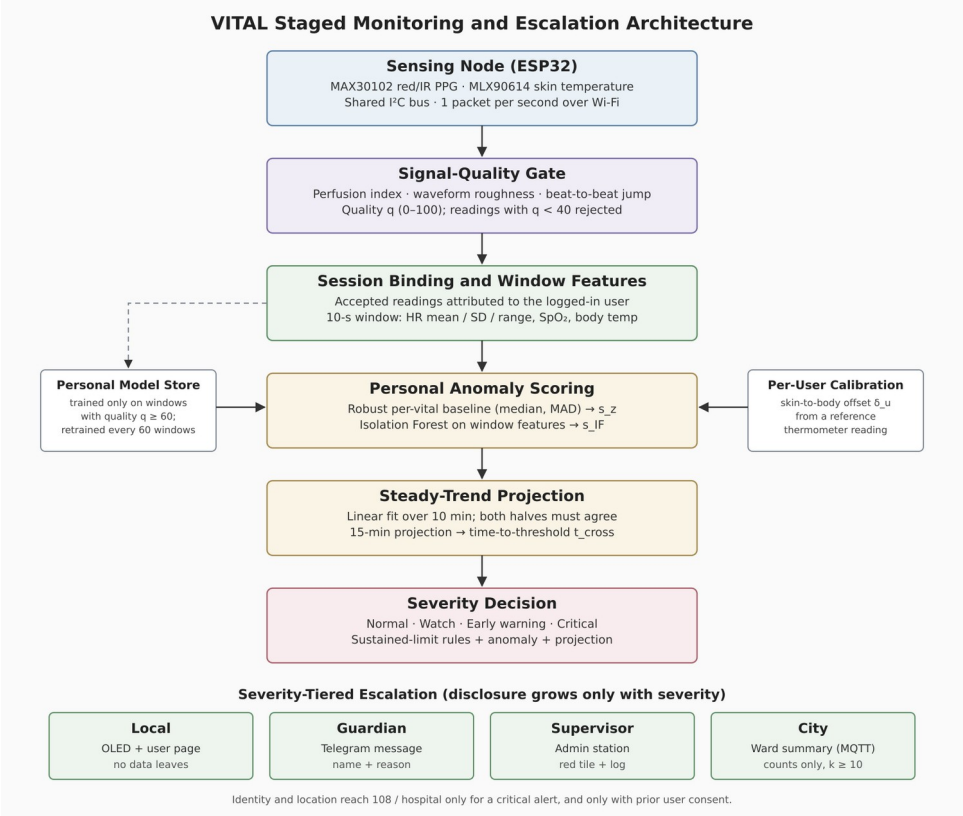


Figure 1. Proposed VITAL staged monitoring and escalation architecture.

All three modules sit on the same I²C bus of the ESP32 (SDA on GPIO 21, SCL on GPIO 22), each with its own address: MAX30102 at 0x57, MLX90614 at 0x5A and the OLED at 0x3C. They are powered from the 3.3 V pin. We run the bus at 100 kHz because that is the fastest the MLX90614 supports. The firmware uses both cores of the ESP32. Core 1 handles sensor sampling, beat detection and the display. Wi-Fi runs as a separate FreeRTOS task on core 0, and the two talk through a queue. This way, a slow network never makes us lose sensor samples.

©VIT IPR&TTCELL

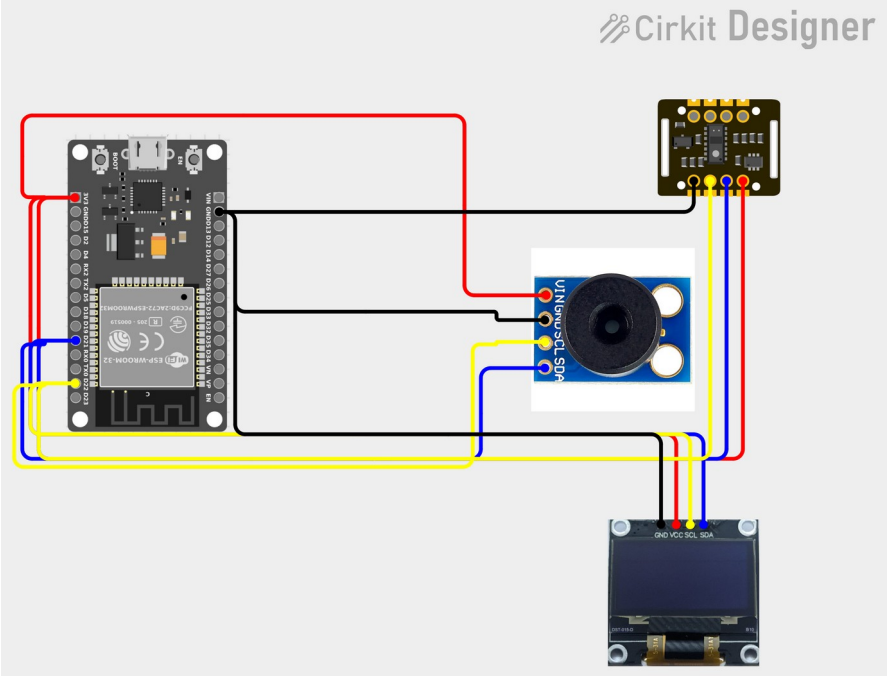


Figure 2. Wiring diagram of the VITAL sensing node (Cirkit Designer).

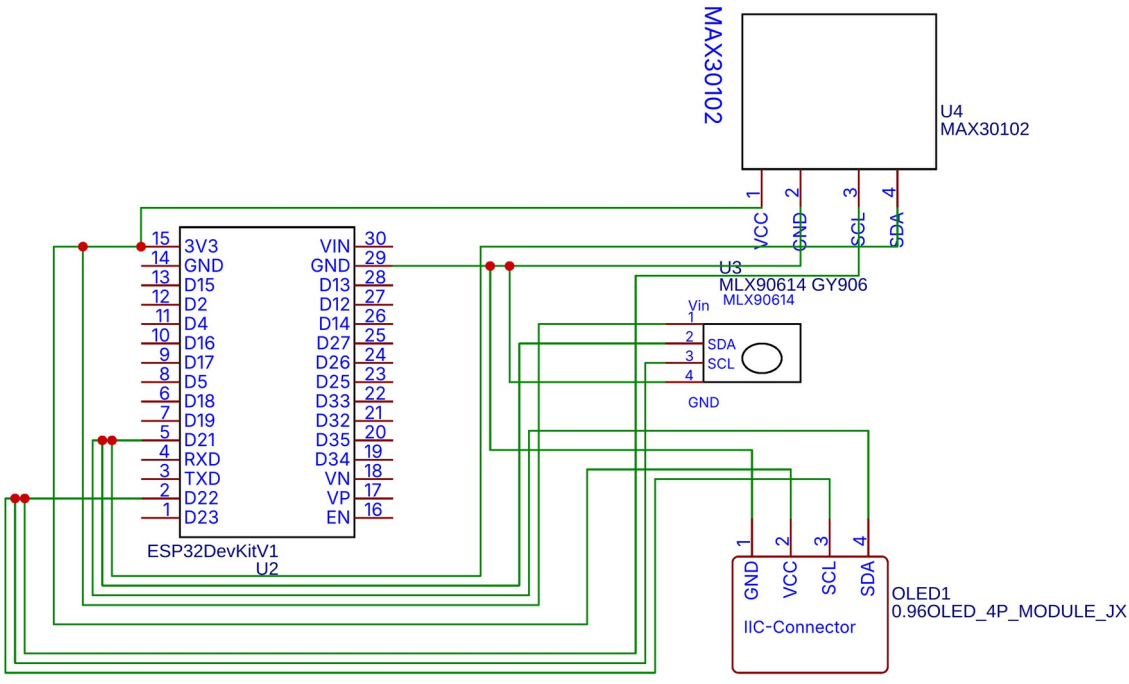


Figure 3. Circuit schematic of the VITAL sensing node (EasyEDA).

7.2 Vital-sign acquisition

With every heartbeat, the amount of blood in the fingertip changes slightly, and so does the amount of light it absorbs. The steady (DC) part of the PPG signal comes from tissue and blood that isn't pulsing. The pulsing (AC) part comes from arterial blood. We get heart rate from the average gap between the last four detected beats:

$$HR = \frac{60}{\Delta t}$$

SpO_2 comes from comparing the pulsing part of the red and infrared signals, each divided by its steady part. The ESP32 calculates this with Maxim's reference algorithm over a 4-second buffer:

$$R = \frac{AC_{red} / DC_{red}}{AC_{ir} / DC_{ir}}$$

The MLX90614 measures skin temperature, which is lower than body temperature. To correct this, each user is calibrated once against a normal thermometer, which gives a personal offset δ_u:

$$T_{body} = T_{skin} + \delta_u, \qquad \delta_u = T_{ref} - \overline{T}_{skin}$$

7.3 Signal-quality gate

For every reading, we look at the last five seconds of the infrared waveform x and compute three things: the perfusion index (how strong the pulse is), how rough the waveform is, and how much the heart rate jumped since the last reading:

$$PI = 100 \cdot \frac{P_{95} - P_5}{\bar{x}}, \qquad \rho = \frac{\sigma(\Delta^2 x)}{P_{95} - P_5}, \qquad |\Delta HR|$$

A logistic-regression model turns these three values into a quality score q from 0 to 100. If q is below 40, the reading is rejected. It is not shown as a valid value, cannot cause an alert and is not saved. To train the personal model we are even stricter and only use windows with an average q of 60 or more. That way, moving fingers and bad placement can't mess up the baseline.

7.4 User attribution and window features

When a user starts a measurement session, the shared device is tied to them, so every accepted reading is saved under that user. Every 10 accepted readings make one window, described by:

$$\mathbf{f} = \left[\overline{HR},\ \sigma_{HR},\ HR_{max} - HR_{min},\ \overline{SpO_2},\ \overline{T}_{body} \right]$$

7.5 Personal anomaly scoring

Each user's model is trained only on their own good-quality windows. It needs at least 30 windows (about five minutes) to start, and it is retrained after every 60 new windows. The model has two parts that cover for each other. The first is a simple but robust baseline for each vital. For heart rate, SpO_2 and temperature, we take the median and the scaled median absolute deviation (MAD). We also set a minimum spread so the model doesn't get too jumpy for very steady users. This tells us how far a value is from the person's normal, in their own units:

$$z_k = \frac{|x_k - m_k|}{\max(1.4826 \cdot MAD_k,\ s_{k,min})}, \qquad s_z = \sigma(1.5\,(z_{max} - 4.5))$$

The second part is an Isolation Forest with 200 trees that looks at all five window features together. It is good at noticing when several vitals have each changed a little at the same time. Its raw score r is scaled using the user's own training scores:

$$s_{IF} = \sigma\left(3 \cdot \frac{r - r_{99}}{r_{99} - r_{50}}\right)$$

Here σ is the logistic function, and r_{99} and r_{50} are the 99th and 50th percentiles of the user's training scores. The final anomaly score for a window is the highest of the three:

$$s = \max\left(s_z,\ s_{IF},\ 0.9 \cdot s_{pop}\right)$$

s_pop is a score based on population limits, so a reading outside the general safe range is always treated as unusual, whatever the personal model says. A new user who doesn't have enough data yet is scored on s_pop alone. We picked the constants using a separate calibration data set, before running the held-out tests in Section 8.

7.6 Steady-trend projection

For heart rate, SpO_2 and temperature, we average the last 10 minutes into 10-second points and fit a straight line y = a + b·t (t in minutes, t = 0 is now). We only consider a warning when three conditions hold. There must be at least two minutes of data. The change projected over the next 15 minutes must be more than a small minimum. And the trend must be steady, which means the older half and the newer half of the window both move the same way, each at least half as fast as the overall slope:

$$\min(d\,b_{old},\ d\,b_{new}) \ \geq\ 0.5\,d\,b$$

d is +1 for a limit that is reached by rising (temperature 38.0 °C, heart rate 120 bpm) and −1 for one reached by falling (SpO_2 92%). If the 15-minute projection crosses the limit L but the current fitted value hasn't yet, the estimated time left is:

$$t_{cross} = \frac{L - a}{b}$$

We added the steadiness check after our first version kept raising false warnings. Without it, a single jump, like the temperature changing when someone walks in from outside, was being stretched into a fake early warning.

7.7 Severity decision and escalation

The rules, the anomaly score and the projection are checked together. A limit rule only counts if the value stays past the limit for 10 seconds, and the same kind of alert is not repeated within five minutes.

Condition	Severity	Escalation and disclosure
All vitals within personal and population ranges	Normal	Local display and user page only
Anomaly score ≥ 0.6, or a vital moderately outside range (e.g. SpO_2 < 95%, HR > 100)	Watch	Plain-language guidance on the user page; no data leaves the hub
Steady trend projected to cross a limit within 15 minutes	Early warning	Guardian Telegram message with estimated time-to-threshold; supervisor log
SpO_2 < 90%, HR > 120 or < 45 bpm, or temperature ≥ 38.5 °C for 10 s; or anomaly ≥ 0.7 in three consecutive windows (not during exertion)	Critical	OLED and user-page alarm, guardian Telegram message, red supervisor tile; with prior consent, name, location and the last five minutes of readings to 108 / hospital
Signal quality q < 40 or no finger detected	Inconclusive	Reading rejected; user asked to reposition the finger; nothing learned

So the severity decides two things at once: what the system does, and how much personal data goes out.

7.8 Privacy-preserving smart-city integration

This part is a design that we have implemented in software but not connected to a real city system. Every VITAL hub (a home, hostel or clinic) sends an hourly summary for its ward to the city platform over MQTT, on the topic city/<city>/health/<ward>/summary. The summary has only counts and averages:

- how many people and readings there were in the ward;
- how many people had a fever (window temperature ≥ 37.8 °C) and how many critical alerts there were; and
- the average heart rate, SpO_2 and temperature.

If a ward has fewer than 10 people, it is merged into the city total so nobody can be picked out. The city platform marks a ward as a possible outbreak cluster when more than 10% of its people have a fever. Names, device IDs and raw signals stay inside the hub. The only exception is a critical emergency the user has agreed to beforehand (Section 7.7). We based this on the consent, purpose-limitation and data-minimisation ideas in India's Digital Personal Data Protection Act, 2023.

©VIT IPR&TTCELL

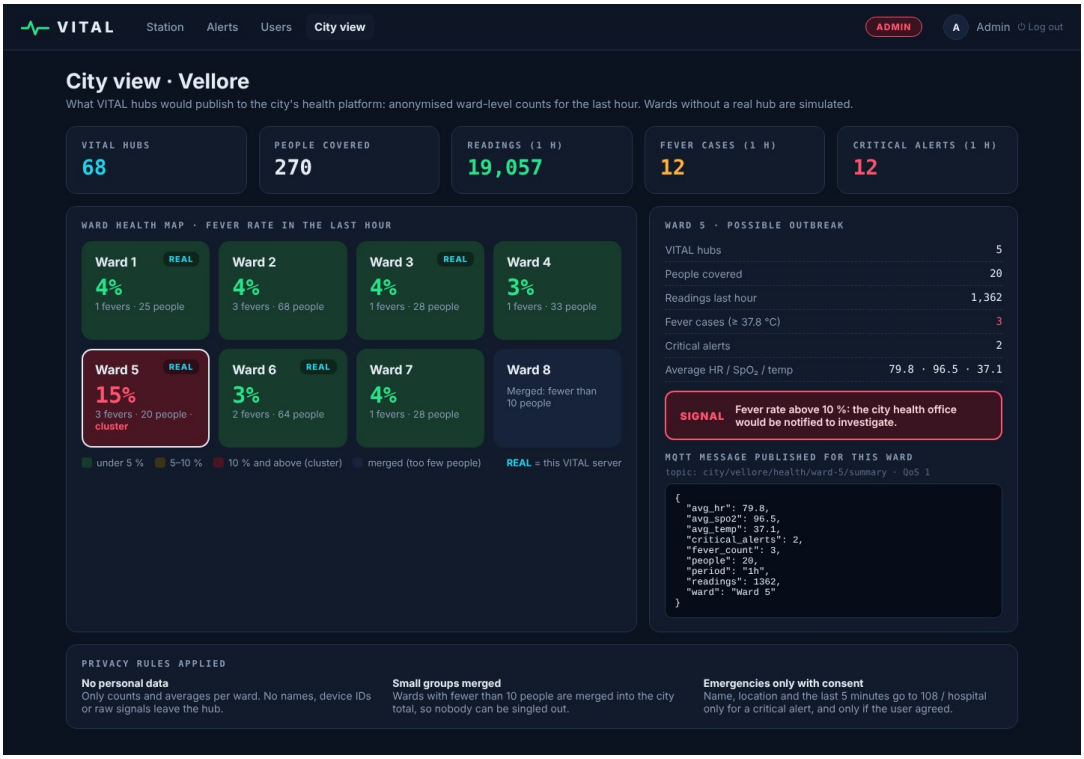


Figure 4. City view: ward fever map and the anonymised MQTT message published for a ward.

7.9 User interfaces

The user page shows the live pulse waveform, the three vitals and short plain-English notes comparing them with the user's own normal. A Simple/Detailed switch opens a detailed view with the raw signals, signal quality, anomaly score, health state and trend projection. The supervisor station shows every user as a live tile. If someone has a critical alert, their tile turns red and shows the reason.



Figure 5. User page during a critical low-SpO_2 alert.

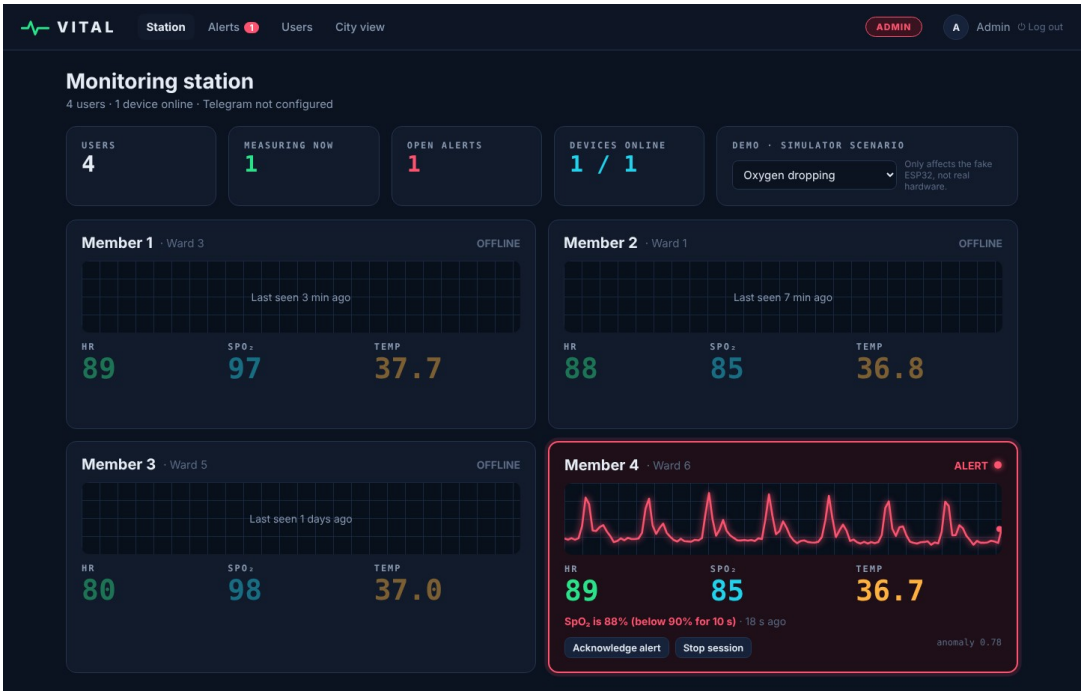


Figure 6. Supervisor station: the affected user's tile turns red with the alert reason.

7.10 Measurement safeguards

To keep the readings and alerts trustworthy, we added these safeguards:

- the infrared level is checked first to make sure a finger is actually on the sensor;
- heart rates that are physically impossible (below 30 or above 220 bpm) are thrown out, and the rest are averaged over four beats;
- the signal-quality gate controls display, alerts and learning;
- a critical condition has to last 10 seconds before it raises an alert;
- anomaly alerts are switched off while the user is exerting themselves (state "Elevated");
- the same kind of alert waits five minutes before it can repeat;
- temperature is calibrated for each user; and
- personal scoring only starts once there is enough personal data.

7.11 Simplified experimental proof of concept

The idea can be shown with a small bench setup: the ESP32, MAX30102, MLX90614 and OLED on a breadboard, plus a laptop running the server. With this setup:

- the firmware measures heart rate, SpO_2 and skin temperature and sends one packet per second;
- the server works out signal quality, the personal anomaly score, the trend projection and the severity;
- alerts appear on the OLED, the user page and the supervisor station, and go to a guardian on Telegram; and
- the device simulator, using the same message format, plays out situations like falling oxygen that can't be safely created in a real person.

7.12 Distinguishing technical mechanism

Put simply, the core of VITAL is one chain that connects reliability, personalisation, prediction and data sharing:

Quality-gated reading → Personal anomaly + steady-trend projection → Severity → Disclosure-scoped escalation

Signal quality controls both the alerts and the learning. Anomalies are judged against the person's own model, not a general one. Only steady trends are projected. And the severity decides both who is told and how much personal data is shared.

©VIT IPR&TTCELL

8. Experimental validation results:

8.1 Software simulation and system-level validation

We tested the actual VITAL code in three ways. First, we tested the machine-learning modules offline on simulated users, using fixed random seeds and the exact same model code that the server runs. Second, we ran the full system end to end: the server, database, alert service and dashboard API all handled live data coming from the device simulator. Third, we checked the ESP32 firmware and the device message format. We tuned the constants on a separate calibration set, so every result below is on data the models had not seen.

Summary of the software tests

Evidence layer	Status	Result
Personal anomaly detection (offline)	PASS	40 simulated users, 180 training windows each; 6,000 held-out healthy windows plus 7,360 abnormal windows.
Trend prediction (offline)	PASS	236 deteriorating and 240 non-deteriorating trajectories; all deteriorations warned before crossing; 0 false warnings.
Signal-quality gate (offline)	PASS	900 held-out waveforms (clean, motion, poor placement).
Health-state classifier (offline)	PASS	1,200 held-out windows, 4 states.
End-to-end system runs (simulator)	PASS	21 real-time runs over 7 scenarios: 6/6 expected critical alerts raised; 3/3 fever early warnings; 0 critical alerts in 6 normal/exercise runs.
ESP32 firmware	PASS (host check)	Both sketches checked against the actual SparkFun MAX3010x and Adafruit MLX90614 / SSD1306 / GFX / BusIO library headers with no errors or warnings. Compilation for the ESP32 target is part of the bench test (8.2).
Device message contract	PASS	Firmware-format packets accepted; wrong device key rejected (HTTP 403); missing temperature sensor handled without failure.

Personal anomaly detection compared with fixed population thresholds (flag at score ≥ 0.7)

Held-out case	Windows	Fixed limits	Isolation Forest only	Personal baseline only	VITAL combined
Healthy held-out windows	6,000	0.0%	0.1%	0.0%	0.1%
Healthy, resting HR ≥ 85 bpm	1,200	0.0%	0.2%	0.0%	0.2%
Personal deviation (population-normal): HR +20 bpm	960	0.0%	2.1%	100.0%	100.0%
Subtle combined: HR +12 bpm and temp +0.5 °C	1,600	13.3%	19.2%	93.8%	94.9%
Tachycardia at rest: HR 125 bpm	1,600	100.0%	1.6%	100.0%	100.0%
Hypoxia: SpO2 87 %	1,600	100.0%	1.9%	100.0%	100.0%
Fever: 38.6 °C	1,600	100.0%	2.7%	100.0%	100.0%

The first two rows show false alarms on healthy windows (lower is better). The rest show how often the problem was caught (higher is better). "Fixed limits" means HR outside 50–100 bpm, SpO_2 below 95% or temperature of 37.5 °C or more.

Trend-prediction results

Measure	Temperature → 38.0 °C	SpO_2 → 92%
Deteriorating trajectories	116	120
Warned before the limit was crossed	116 (100%)	120 (100%)
Median warning lead time	5.3 min	5.9 min
Lead time exceeded in 90% of cases	2.8 min	3.8 min
Median error of estimated time-to-threshold	0.4 min	0.6 min
False warnings: stable temperature	0 / 120	-
False warnings: one-off temperature jump (+0.3 to +0.6 °C)	0 / 120	-

Signal-quality gate and health-state classifier

Held-out waveform type	Waveforms	Accepted (q ≥ 40)
Clean pulse	300	292 (97.3%)

©VIT IPR&TTCELL

Held-out waveform type	Waveforms	Accepted (q ≥ 40)
Moving finger (motion artefacts)	300	47 (15.7%)
Poor placement (weak pulse)	300	57 (19.0%)
Health-state classifier, held-out windows	1,200	accuracy 100.0%

End-to-end system runs (server + device simulator, real time)

Scenario	Runs	Expected	Observed	Readings accepted
Normal resting	3	no alert	0 critical alerts; 0 warnings	100%
After exercise (HR ≈ 112)	3	no critical alert	0 critical alerts (anomaly suppressed during exertion); 1 heart-rate trend warning in 3 runs	99%
Oxygen dropping	3	critical alert	3/3 critical; first alert after mean 40 s, raised by the personal-anomaly path before SpO_2 fell below 90% in every run	100%
High HR at rest (≈ 134)	3	critical alert	3/3 critical (heart-rate rule); mean 19 s after onset	96%
Developing fever	3	early warning	3/3 early warning, mean 103 s; 3/3 critical from the personal-anomaly path (mean 60 s) while temperature was below 38 °C	100%
Moving finger	3	readings rejected, no alert	0 critical alerts; 0 warnings	3%
No finger	3	nothing stored	0 critical alerts; 0 warnings	0%

What the results mean: We ran the whole pipeline on simulated users, both healthy ones and ones getting worse. In the offline tests, the personal baseline caught every personal change that still looked "normal" to fixed limits, and the combined score kept false alarms on healthy windows well under one percent. On its own, the Isolation Forest was poor at catching one vital drifting far away, but it helped with several vitals changing a little together, which is why we kept both parts. The steadiness check got rid of false warnings from one-off jumps, and every steady decline was still warned several minutes early. In the live system runs, the personal-anomaly path raised the first alert in the oxygen-drop and fever cases before any population limit was crossed. The quality gate threw out almost all moving-finger readings, and there were no critical alerts in the normal or exercise runs. There was one heart-rate trend warning during exercise, so long periods of exertion can still trigger a trend warning. These are simulation results under our own models, not clinical accuracy. We are aware of some limitations: the users and waveforms are simulated, the health-state classifier was trained and tested on generated examples (its 100% just means the synthetic classes are easy to separate), and real finger readings will be noisier than the simulator.

8.2 Physical laboratory bench validation

[To be completed after the bench test.] Our physical prototype has an ESP32 DevKit V1, a MAX30102 module, a GY-906 MLX90614 module and a 0.96-inch SSD1306 OLED on a breadboard. I²C runs on GPIO 21 (SDA) and GPIO 22 (SCL) at 100 kHz, and the board talks to the laptop server over Wi-Fi. We first run a sensor-test sketch to confirm all three I²C devices are found, then load the VITAL firmware, which sends one packet per second. We compare the readings with a regular fingertip pulse oximeter and a clinical thermometer.

[Insert photograph of the bench prototype]	[Insert close-up of the ESP32, sensors and OLED]

Figure 7. Physical bench setup of the VITAL prototype.

Figure 8. Close-up of the sensing node.

Observed parameter	Reference device	VITAL	Difference
I²C devices detected (0x3C, 0x57, 0x5A)	-	[ ]	-
Heart rate at rest (bpm)	[ ]	[ ]	[ ]
SpO_2 at rest (%)	[ ]	[ ]	[ ]
Temperature after calibration (°C)	[ ]	[ ]	[ ]
Packet delivery to server (packets / min)	60	[ ]	[ ]
OLED shows readings and ALERT from server	-	[ ]	-

Scope of the physical test: we do not create dangerous conditions (low SpO_2, fever, very high heart rate) in any person. Those cases are tested with the simulator in 8.1, which sends exactly the same message format as the real device.

8.3 Combined validation conclusion

Putting it together, we have held-out tests of the anomaly detection, trend prediction, signal-quality and health-state modules; live end-to-end runs of the full server, alert and dashboard pipeline using the simulator; and a check of the ESP32 firmware against the real sensor libraries. The software tests show how the full monitoring and alert logic behaves under our models, and the bench test in 8.2 checks the sensing side on real hardware. For now, we place the invention at TRL 3 (Experimental proof of concept). Once the bench test in 8.2 is done, it can move up to TRL 4 (Technology validated in a lab).

9. What aspect(s) of the invention need(s) protection?

We are mainly looking to protect the staged monitoring method and the way its stages depend on each other. The hardware itself (microcontroller, PPG sensor, thermometer, display) is standard and not what we are claiming.

1. **Quality-gated learning: A quality score for each reading decides whether it is shown, whether it can raise an alert, and whether it is used to train the user's personal model.**
2. **Personal models on a shared device: Readings from one shared device are linked to whoever is logged in, and each user gets a separate personal model.**
3. **Two-part personal scoring: Each window is checked against a robust per-vital baseline (median and MAD with a minimum spread) and an Isolation Forest scaled by the user's own scores, with population limits as a safety floor.**
4. **Steady-trend time-to-threshold: A trend is only projected when the older and newer halves of the window agree in direction and rate, and the time left before a danger limit is estimated.**
5. **Severity-based escalation: The severity picks both the alert level (local, guardian, supervisor, emergency services) and how much personal data is shared at that level.**
6. **Privacy-preserving city summaries: Ward-level summaries, with small wards merged, are sent to a smart-city platform, and identity is released only for critical alerts the user has agreed to.**
7. **Per-user temperature calibration: Body temperature is calculated from contactless skin temperature using an offset calibrated for each user against a normal thermometer.**

Core protectable relationship:

A quality decision on every reading controls both alerts and personal-model learning on a device shared by several users. Each window is scored against the user's own robust baseline and multivariate model, only steady trends are projected to estimate the time left, and the resulting severity decides both who is alerted and how much personal data is shared, all the way up to anonymised ward-level smart-city reporting.

10. What is Technology readiness level of your invention? (Tick the appropriate TRL)

Research			Development			Deployment		
TRL 1	TRL 2	TRL 3	TRL 4	TRL 5	TRL 6	TRL 7	TRL 8	TRL 9

©VIT IPR&TTCELL

Basic Principles observed	Technology concept formulated	Experimental proof of concept	Technology validated in a lab	Technology validated in a relevant environment (industrially relevant in case of key enabling technologies)	Technology demonstrated in a relevant environment (industrially relevant in case of key enabling technologies)	System prototype demonstration in an operational environment	System complete and qualified	Actual system proven in an operational environment (competitive manufacturing in case of key enabling technologies, or in space)
		✓						

Selected TRL: TRL 3 — Experimental proof of concept.

----------END OF THE DOCUMENT----------